Generador offline de Quiz + Presentación desde PDF

> **Resumen en una línea**: una aplicación **100 % local** (sin nube) que toma un PDF académico y genera, con IA local vía **Ollama**, (1) una presentación `.pptx` con plantilla universitaria y (2) un Quiz de opción múltiple interactivo con exportación a JSON y PDF imprimible.

Este documento es el **único punto de verdad** del proyecto. Cubre:

- Qué hace la aplicación y por qué existe.
- La arquitectura completa y las decisiones de diseño tomadas.
- Cada archivo, cada función pública y el flujo de datos entre ellos.
- El historial de mejoras implementadas y las razones detrás de cada una.
- Cómo instalar, ejecutar, depurar y extender el sistema.

> **Contrato de mantenimiento**: *toda* modificación futura del proyecto debe quedar reflejada en este README (nueva funcionalidad, cambio de decisión, nueva dependencia, nueva mejora). La sección Historial de mejoras es un changelog técnico ordenado cronológicamente que se amplía con cada iteración.

Tabla de contenidos

1. Motivación y principios de diseño
2. Vista rápida del sistema
3. Arquitectura del pipeline
4. Estructura de archivos
5. Módulos y API interna
6. Modelos de datos y contratos
7. Prompts
8. Instalación y ejecución
9. Historial de mejoras (changelog técnico)
10. Decisiones de diseño y *por qué* no se hizo de otra forma
11. Roadmap pendiente
12. Convenciones de contribución y del README

---

1. Motivación y principios de diseño

El objetivo es cubrir un caso de uso académico muy concreto: a partir de apuntes o papers en PDF, obtener **material docente listo para usar** (una presentación profesional y un quiz de autoevaluación) sin que la información salga nunca de la máquina del usuario.

Principios innegociables del proyecto:

Principio	Implicación práctica
Privacidad total	Cero llamadas a servicios cloud. Inferencia con Ollama local, procesado de PDF con PyMuPDF local, generación de PPTX/PDF local.

Principio	Implicación práctica
Reproducibilidad	Temperaturas bajas (0.2–0.4), salidas validadas con Pydantic, semillas de diseño (distribución Bloom por rangos) para minimizar varianza.
Calidad > cantidad	Preferimos 8 preguntas excelentes a 20 mediocres. El revisor regenera lo malo en vez de inundar con más contenido.
Anclaje al documento	El LLM nunca genera contenido "de memoria". Todo pasa por una KnowledgeBase intermedia que actúa de fuente única de verdad.
Código modular y tipado	Pydantic + `Literal` para vocabularios cerrados, `dataclass` para estructuras internas, `from __future__ import annotations` en todos los módulos.
Fallo explícito y recuperable	Jerarquía propia de excepciones (`src/exceptions.py`), reintentos con backoff, degradación controlada (p. ej. regeneración selectiva de preguntas problemáticas).

2. Vista rápida del sistema

```
┌──────────────┐      ┌──────────────────┐      ┌──────────────────┐
│   PDF (↑UI)  │──▶   │ PDF→Markdown     │──▶   │ Map-Reduce LLM   │
└──────────────┘      │ (pymupdf4llm)    │      │   + Extractores  │
                      └──────────────────┘      └──────────────────┘
                                                          │
                                                          ▼
                                              ┌──────────────────────┐
                                              │  KnowledgeBase (JSON)│◀── fuente
única
                                              └──────────────────────┘      de verdad
                                                          │
                                  ┌───────────────────────┴──────────────────────────┐
                                  ▼                                                  ▼
        ┌──────────────────────┐
┌──────────────────────┐
        │  SlidePlan (LLM)     │                                  │  QuizPlan (LLM)
│
        │  + sanitización      │                                  │  + sanitización
│
        └──────────┬───────────┘
└──────────┬───────────┘
                   │                                                     │
                   ▼                                                     ▼
        ┌──────────────────────┐
┌──────────────────────┐
        │ Bullets por slide    │                                  │ Pregunta por
pregunta│
        │ (1 llamada por slide)│                                  │ (1 llamada por
preg.)│
        └──────────┬───────────┘
└──────────┬───────────┘
                   │                                                     │
                   ▼                                                     ▼
```

```
        ┌──────────────────────┐
┌──────────────────────┐
        │ Revisor slides       │   <─── refinamiento  ──>       │ Revisor quiz
│
        │ (determinista + LLM) │                                │ (determinista +
LLM)│
        └──────────┬───────────┘
└──────────┬───────────┘
                   │                                                  │
                   ▼                                                  ▼
        ┌──────────────────────┐
┌──────────────────────┐
        │ Render PPTX          │                                │ Quiz
interactivo    │
        │ (python-pptx)        │                                │ + PDF
(reportlab)   │
        └──────────────────────┘
└──────────────────────┘
```

Tres pasos mentales para el usuario:

1. Subir PDF → el backend lo convierte a Markdown y lo resume a una KnowledgeBase estructurada.
2. Pulsar *Generar Quiz* y/o *Generar Presentación* → se ejecuta el pipeline correspondiente.
3. Previsualizar, responder el quiz en la propia UI y descargar artefactos (`.pptx`, `.json`, `.pdf`).

---

3. Arquitectura del pipeline

El pipeline está diseñado como una **cadena de contratos Pydantic**. Cada etapa tiene *input* y *output* formalmente validados, lo que permite:

- Detectar drift del LLM al instante y reintentar solo el paso fallido.
- Testar cualquier etapa de forma aislada con datos sintéticos.
- Sustituir el LLM por otro sin tocar el resto del código.

3.1. Fases del pipeline (orden cronológico)

#	Fase	Entrada	Salida	Módulo
1	Extracción PDF	bytes	`ProcessedPDF` (Markdown enriquecido)	`src/pdf_processor.py`
2	Chunking por headings	Markdown	`list[Chunk]`	`src/map_reduce.py`
3	Extractores literales	Markdown	`LiteralHints`	`src/extractors.py`
4	**MAP** (resumen por chunk)	`Chunk`	resumen parcial en texto	`src/map_reduce.py`

#	Fase	Entrada	Salida	Módulo
5	**REDUCE → KB**	resúmenes parciales + hints	`KnowledgeBase` (JSON)	`src/map_reduce.py` + `src/knowledge_base.py`
6a	Plan de slides	`KnowledgeBase`	`SlidePlan`	`src/pptx_generator.py` + `src/plans.py`
6b	Plan de quiz	`KnowledgeBase`	`QuizPlan`	`src/quiz_generator.py` + `src/plans.py`
7a	Redacción de bullets	`PlannedSlide` + KB	`BuiltSlide`	`src/pptx_generator.py`
7b	Redacción de preguntas	`PlannedQuestion` + KB	`QuizQuestion`	`src/quiz_generator.py`
8	Revisor + refinamiento	artefacto + KB	artefacto pulido	`src/critics.py`
9a	Render PPTX	`PresentationPlan` + plantilla	bytes `.pptx`	`src/pptx_generator.py`
9b	Render Quiz PDF	`Quiz`	bytes `.pdf`	`src/quiz_pdf_exporter.py`
10	UI + descarga	artefactos	navegador	`app.py`

3.2. Por qué este diseño y no un único prompt

El enfoque *naïve* sería: "pon todo el PDF en un prompt enorme y pide 10 preguntas y una presentación". Rechazamos este enfoque por cinco razones:

1. **Context window**: modelos locales de 7-9 B tienen 4k-8k tokens útiles. Un PDF de 50 páginas los sobrepasa.
2. **Coherencia global**: una sola pasada tiende a repetir conceptos, olvidar otros y mezclar tablas.
3. **Control de calidad**: con contenido intermedio estructurado (KB, planes) podemos *auditar* y *regenerar* partes concretas.
4. **Reutilización**: la KB sirve a la vez para quiz y para slides; no se recalcula.
5. **Depuración**: cuando algo falla, se ve exactamente en qué etapa.

---

4. Estructura de archivos

```
tfg-local/
├── app.py                          # UI Streamlit (entrada principal)
├── plantilla_universidad.pptx      # Plantilla corporativa con layouts 0
(portada) y 2 (contenido)
├── requirements.txt                # Dependencias con versiones mínimas
├── README.md                       # Este documento
├── .streamlit/
│   └── config.toml                 # Tema y maxUploadSize (50 MB)
```

```
└── src/
    ├── __init__.py                # Metadatos del paquete (__version__)
    ├── config.py                  # Constantes globales (modelos, límites,
layouts…)
    ├── exceptions.py              # Jerarquía de excepciones de dominio
    ├── ollama_client.py           # Wrapper de Ollama con reintentos y preflight
    ├── pdf_processor.py           # PDF → Markdown + detección escaneos y tablas
    ├── extractors.py              # Heurísticas literales (defs, código,
fórmulas, términos)
    ├── knowledge_base.py          # Pydantic de la KB canónica del documento
    ├── map_reduce.py              # Chunking + fase MAP + fase REDUCE → KB
    ├── plans.py                   # SlidePlan / QuizPlan + saneadores con fuzzy
matching
    ├── prompts.py                 # Prompts LLM centralizados (todos en español)
    ├── quiz_generator.py          # Pipeline del quiz (plan + 1 llamada por
pregunta)
    ├── pptx_generator.py          # Pipeline de slides (plan + 1 llamada por
slide + render)
    ├── critics.py                 # Revisor determinista + LLM + refinamiento
selectivo
    └── quiz_pdf_exporter.py       # Export del quiz a PDF imprimible (reportlab)
```

---

5. Módulos y API interna

A continuación, una descripción por módulo de **todo lo público** que expone y **el porqué** de cada decisión. Está ordenado igual que el pipeline.

5.1. `src/config.py`

Constantes inmutables (`Final`) que controlan el sistema entero. Separar configuración del código evita magia dispersa.

Constante	Valor	Propósito
`OLLAMA_BASE_URL`	`http://localhost:11434`	Endpoint Ollama.
`AVAILABLE_MODELS`	tupla de tags completos	Los modelos que el usuario puede elegir en la UI.
`DEFAULT_MODEL`	`qwen2.5:7b`	Compromiso calidad/velocidad para GPU de 8-12 GB VRAM.
`LLM_TEMPERATURE`	`0.2`	Baja por defecto para estabilidad. La redacción de preguntas sube a 0.4 para un poco de creatividad.
`NUM_CTX`	`8192`	Ventana usable en 8-12 GB VRAM.
`CHUNK_SIZE_CHARS`	`3000`	~750 tokens por chunk, deja margen al prompt.

Constante	Valor	Propósito
`CHUNK_OVERLAP_CHARS`	`250`	Evita cortar un concepto entre dos chunks.
`MAX_INPUT_PAGES` / `MAX_INPUT_CHARS`	`50` / `50_000`	Límite duro anti-pdf-bomba.
`DEFAULT_NUM_QUESTIONS` / `MAX_NUM_QUESTIONS`	`10` / `30`	Rango expuesto en la UI.
`LAYOUT_TITLE` / `LAYOUT_CONTENT`	`0` / `2`	Índices verificados contra `plantilla_universidad.pptx`. El layout 1 (section header) **no** tiene body y no sirve para contenido.
`MAX_BULLETS_PER_SLIDE`	`5`	Regla pedagógica clásica.
`MAX_CHARS_PER_BULLET`	`180`	~2 líneas a 20 pt en esta plantilla. Es el umbral que se comunica al LLM y el que usa `_trim_by_words`.
`MAX_CHARS_SLIDE_TITLE`	`80`	Se recorta duro con `…` si se excede.
`DEFAULT_NUM_SLIDES_MIN/MAX`	`6` / `14`	Rango razonable para una clase de ~20 min.

5.2. `src/exceptions.py`

Jerarquía propia que permite a la UI (`app.py`) discriminar el tipo de fallo y mostrar mensajes accionables.

```
AppError
├── PDFError
│   ├── ScannedPDFError
│   └── PDFTooLargeError
├── OllamaError
│   ├── OllamaUnavailableError
│   └── OllamaModelNotFoundError
├── GenerationError       # error de contenido (plan vacío, JSON inválido…)
└── TemplateError         # plantilla .pptx inexistente o corrupta
```

5.3. `src/ollama_client.py`

Wrapper sobre `ollama-python` con:

- `preflight()` → verifica servidor (`/api/tags`) y modelo instalado. **Match exacto por tag** (`qwen2.5:7b`, no `qwen2.5`) porque `/api/generate` exige el nombre completo; un match laxo pasaría el preflight y luego fallaría con 404.
- `generate(prompt, system, json_mode, temperature)` con reintentos exponenciales (3 intentos) vía `tenacity` sobre errores de red/respuesta.

- `generate_json(...)` con limpieza defensiva: elimina ```` ```json ```` fences, recorta al primer `{`/`[` y al último `}`/`]`, reporta JSONDecodeError con los primeros 400 caracteres de la respuesta para diagnóstico.
- Errores de infraestructura se mapean a `OllamaUnavailableError` / `OllamaModelNotFoundError`.

5.4. `src/pdf_processor.py`

Conversión PDF → Markdown con PyMuPDF y pymupdf4llm. Aporta:

- **Detección de escaneos** (`_detect_scanned`): si menos del 30 % de páginas tienen ≥40 caracteres extraíbles, se lanza `ScannedPDFError` con sugerencia de OCR.
- **Extracción de pies de imagen** (`_extract_image_context`): para cada imagen se busca el bloque de texto más cercano verticalmente como caption; se añade al final del Markdown bajo `## Contexto de imágenes`. Da al LLM contexto visual sin depender de visión por computador.
- **Detección de tablas** (`_looks_like_tables`): busca líneas separadoras GFM (`---|---`) en el markdown resultante.
- **Límites duros**: páginas (`MAX_INPUT_PAGES`) y caracteres (`MAX_INPUT_CHARS`).

Salida: `ProcessedPDF(markdown, num_pages, num_chars, num_images, has_tables)`.

5.5. `src/extractors.py`

Heurísticas puras (sin LLM) para localizar **material literal** del documento. Se ejecutan antes del REDUCE y se inyectan como material de anclaje:

- `extract_literal_definitions`: tres patrones regex (`"X es Y"`, `"Se llama/define X a Y"`, `"**Término**: definición"`). Filtra sintagmas término de >6 palabras y definiciones de ❤️ palabras.
- `extract_code_fences`: bloques ``` con lenguaje si se declaró. Límite 1500 chars por bloque, máx. 10.
- `extract_formulas`: líneas con `=` y al menos un símbolo matemático (`[\d\+\-\*/\^±√π∑∏∞≤≥≠∈∉⊂⊃∪∩∂Δ∇α-ω]`). Descarta `key=value` de >18 palabras.
- `extract_key_terms`: negritas Markdown + títulos; excluye “Contexto de imágenes”, “Índice”, “Indice”.

Todo va a `LiteralHints` con un `to_prompt_block()` que serializa de forma segura (corta por línea si se pasa de `max_chars=3500`).

5.6. `src/knowledge_base.py`

Define los **contratos Pydantic** de la KB. Es el nudo del proyecto: una vez hay una KB válida, el resto es determinista.

Tipos de átomo:

Clase	Prefijo id	Campos destacados
`Definition`	`def:*`	`term`, `definition`, `subtopic`, `verbatim` (si es cita literal del extractor).
`Example`	`ex:*`	`name`, `description`, `attributes[]`, `methods[]`, `subtopic`.
`FormulaOrCode`	`fc:*`	`kind` (`formula`/`code`), `content`, `caption`, `language`.
`NumericDatum`	`dt:*`	`value` (como string, p. ej. `"42%"`), `description`.
`Relation`	`rel:*`	`kind` (libre), `source`, `target`, `description`.

Normalización de ids (`_normalize_id`, aplicado en `@field_validator(..., mode="before")`):

- Extrae el prefijo correcto aunque el LLM devuelva otro.
- Descompone acentos (`NFKD`), pasa a minúsculas, `ñ → n`.
- Sustituye cualquier carácter no `[a-z0-9_-]` por `_`.
- Recorta a 40 caracteres.
- Evita ids vacíos (`"item"` de fallback).

Esta normalización se aplica **antes** de que Pydantic compruebe el `pattern`, de forma que el esquema final puede permanecer estricto sin rechazar ids con tildes.

Métodos útiles de `KnowledgeBase`:

- `atom_count`, `atom_ids()`, `get_atom(id)`, `atoms_by_subtopic()`.
- `to_markdown()` → render humano.
- `to_prompt_context(max_chars=…)` → serialización segura para prompts downstream (trunca de forma estable).
- `slugify_id(prefix, text)` → utilidad libre para construir ids estables en tests o fallbacks.

5.7. `src/map_reduce.py`

Implementa el pipeline **MAP → REDUCE → KB**:

- `split_markdown(md)`: divide por cabeceras (`#…######`) preservando el encabezado, después empaqueta secciones en chunks `<= CHUNK_SIZE_CHARS`. Si una sección supera el tamaño, parte por párrafos; si un párrafo es aún mayor (p. ej. una tabla gigante), trocea por longitud. Finalmente añade `CHUNK_OVERLAP_CHARS` de cola del chunk anterior al siguiente.
- `_map_phase`: una llamada LLM por chunk con `MAP_SUMMARY_PROMPT`. Emite progreso vía `ProgressCallback`.
- `_reduce_to_kb`: consolida los resúmenes + `LiteralHints` con `REDUCE_TO_KB_PROMPT` en **modo JSON**. Construye `KnowledgeBase(**raw)`; si Pydantic falla, se eleva `GenerationError` con los 2 primeros errores para diagnóstico.
- `build_knowledge_base(client, markdown, progress_cb=...)` — **API pública**: orquesta chunking + hints literales + map + reduce.
- `consolidate_document(...)`: *shim* de compatibilidad que devuelve la KB serializada como Markdown. Mantenido por si algún consumidor legacy aún espera texto plano.

5.8. `src/plans.py`

Contratos intermedios entre la KB y la generación final. Son los que evitan que el LLM haga “todo en un shot”.

5.8.1. SlidePlan

- `SlideKind = Literal["intro", "definition", "example", "comparison", "code", "process", "relations", "outlook", "conclusion"]`.
- `PlannedSlide(title, kind, atom_ids, focus)`. `atom_ids` deduplica automáticamente.
- `SlidePlan(presentation_title, slides)` con 3-20 slides.

5.8.2. QuizPlan

- `BloomLevel = Literal["recordar", "comprender", "aplicar", "analizar", "evaluar", "crear"]`.
- `QuestionKind = Literal["definicion", "diferenciacion", "caso_practico", "comparacion", "analisis_consecuencia", "juicio_alternativas", "completar_codigo"]`.
- `BLOOM_RECOMMENDED_KINDS`: mapa orientativo Bloom → kinds recomendados (inyectado en el prompt).
- `PlannedQuestion(id, bloom_level, concept_id, kind, focus)`.
- `QuizPlan.questions` reasigna ids 1..N en el validator para que siempre sean consecutivos.

5.8.3. Fuzzy matching de ids (reconciliación LLM ↔ KB)

El LLM, al planificar, escribe a menudo ids levemente distintos a los de la KB: `def:Bicicleta` (mayúscula), `rel:subclase_de` (slug corto frente al `rel:subclase_de_bicicletademontana_bicicleta` real), `ex:farmaceutica` frente a `ex:farmacéutica` (tildes). Para no descartar esas referencias y perder preguntas/slides:

- `_deaccent_lower`, `_normalize_slug` (elimina tildes, colapsa plurales `-s`/`-es` de palabras >3 chars), `_split_id`, `_collapse` (quita `_` y `-`).
- `resolve_atom_id(raw, valid_ids) -> str | None`: cascada de intentos (exacto → normalizado → prefijo `A_` ⊂ `B` → colapsado con inclusión → Jaccard de tokens ≥ 0.6). **Restricción clave**: solo considera candidatos con el **mismo prefijo** (`def:` → `def:`, `rel:` → `rel:`), porque el resto del pipeline asume el tipo del átomo (kind del slide/pregunta).
- `_reconcile_atom_ids(list, valid_ids) -> (resolved, lost)`.

5.8.4. Saneadores

- `sanitize_slide_plan(plan, kb)`:
  - Descarta slides con `kind=conclusion` **o** título normalizado que empieza por `"conclus"` (la conclusión final la añade el renderer de forma determinista → se evita duplicar).
  - Reconcilia `atom_ids` con fuzzy match; registra en log cada id recuperado o perdido.
  - Descarta slides que se queden sin átomos, salvo `intro`/`outlook` (admiten cero átomos por diseño).
  - Si el plan queda vacío, devuelve el original sin cambios (mejor que petar el pipeline entero).
- `sanitize_quiz_plan(plan, kb, target_count)`:
  - Reconcilia `concept_id` con fuzzy match.
  - Deduplica por tupla `(concept_id, bloom_level, kind)`: un mismo concepto puede aparecer varias veces si cambia el nivel o el formato de pregunta (esto es deseable pedagógicamente).
  - Recorta al `target_count`, reasigna ids 1..N.
- `bloom_distribution(plan) -> {BloomLevel: int}`: utilidad de diagnóstico.

5.9. `src/quiz_generator.py`

Pipeline en dos pasos, uno a uno:

1. **`plan_quiz(client, kb, num_questions)`** → pide `QuizPlan` con `QUIZ_PLAN_PROMPT`, normaliza alias (`bloom`/`bloom_level`, kinds en inglés), construye Pydantic, sanea.
2. **`generate_single_question(client, kb, planned, previous)`** → usa `QUIZ_QUESTION_PROMPT` con:
   - `concept_detail` renderizado por `_atom_markdown(atom)` (el átomo central).

    - `related_context`: otros átomos del mismo subtopic priorizando `Example` y `Relation` (hasta 8) — son el mejor material para distractores plausibles.
    - `previous_questions`: resumen de las últimas 5 preguntas ya redactadas para evitar solape.
    - Temperatura 0.4 (un pelín más de creatividad que la planificación a 0.2).
3. **`generate_quiz(client, kb, num_questions, refine=True)`** → pipeline completo. Tras generar todas las preguntas, si `refine=True`, delega en `critics.refine_quiz` (import diferido para romper el ciclo). Preserva `planned.id` durante la generación para que el revisor pueda mapear, y al final reasigna ids consecutivos 1..N para la UI.

`QuizQuestion` tiene un validador `correct_answer: Literal["A","B","C","D"]` y un normalizador (`_normalize_letter`) tolerante a `" a "`, `"a."`, etc.

5.10. `src/pptx_generator.py`

Pipeline de presentación en tres capas:

- **Plan** (`plan_slides`): pide el `SlidePlan` con `SLIDE_PLAN_PROMPT`, sanea contra la KB, recorta a rango razonable, trunca títulos con `_truncate_title`.
- **Redacción** (`render_slide_bullets`): una llamada por slide con `SLIDE_BULLETS_FROM_ATOMS_PROMPT` incluyendo:
    - Los átomos asignados a esa slide (`_atoms_for_slide`).
    - El índice completo para que el LLM *sepa* qué no debe repetir.
    - El `kind` y el `focus` narrativo.
- **Limpieza del texto** (`_clean_bullet`): tras el LLM aplicamos una función de calidad que descarta bullets con:
    - `…` o `...` (síntoma de truncamiento LLM; se descarta entero, preferimos regenerar antes que mostrar cortado).
    - Prefijos `def:` / `ex:` / `fc:` / `rel:` / `dt:` (copia del bloque de contexto).
    - Patrón `· definición ·`, `· ejemplo ·`, etc. (también copia del contexto).
    - Aperturas meta (`"en esta diapositiva"`, `"a continuación"`, `"se habla de"`, ...).
    - Longitud <25 chars (bullets triviales).
    - Bullets demasiado largos: se recortan por palabra con `_trim_by_words` (**sin** puntos suspensivos), añadiendo punto final si hace falta.
    - Si **todos** los bullets se descartan → `GenerationError` → el revisor marca la slide como crítica y dispara regeneración.
- **Conclusión** (`_render_conclusion`): misma lógica con `CONCLUSION_FROM_KB_PROMPT`.
- **Render PPTX** (`render_pptx`):
    - Portada con `LAYOUT_TITLE`.
    - Slide de índice construida dinámicamente **a partir de las slides que sobrevivieron** a la limpieza (no del plan original → consistencia visible).
    - Una slide por cada `BuiltSlide` con bullets (las vacías se omiten).
    - Slide de conclusiones si existen.
    - `auto_size = TEXT_TO_FIT_SHAPE`, fuente 20 pt por defecto, `word_wrap=True`.
- **API pública** (`generate_presentation`): orquestador KB → plan → bullets → revisor → conclusión → render PPTX.

5.11. `src/critics.py`

Revisor con dos pasadas (**determinista** + **LLM**) y regeneración selectiva. El módulo evita ciclos de import con `TYPE_CHECKING` y con imports diferidos dentro de las funciones.

5.11.1. Grounding

- `_tokens(text)`: tokeniza, quita stopwords en español, filtra <4 chars.
- `_kb_vocabulary(kb)`: conjunto de tokens relevantes de la KB (main_topic + subtopics + contenido de todos los átomos).
- `_is_grounded(text, vocab, min_hits=1)`: True si el texto comparte ≥1 token con el vocabulario.

Esto permite detectar cuándo el LLM se inventa un bullet o una pregunta sin raíz en el documento.

5.11.2. Reglas deterministas del quiz (`_deterministic_quiz_issues`)

Kind	Qué detecta	Severidad
`banned_phrase`	Opciones "Todas las anteriores", "A y B", "Ninguna es correcta"...	`high`
`unbalanced_options`	Una opción ≥ 2.2× más larga que el promedio del resto (pista involuntaria).	`medium`
`meta_language`	Enunciado/justificación con "según el texto", "el autor dice", etc.	`medium`
`not_grounded`	Pregunta + correcta + justificación sin tokens de la KB.	`medium`
`duplicate`	Stem (8 tokens más significativos) coincidente con otra pregunta.	`medium`

5.11.3. Reglas deterministas de slides (`_deterministic_slide_issues`)

Kind	Qué detecta	Severidad
`meta_language`	Bullets con "en esta diapositiva", "se estudia...", etc.	`medium`
`too_shallow`	<2 bullets en una slide que no es intro/outlook/conclusion.	`medium`
`not_grounded`	Bullets sin tokens de la KB (en slides que no son intro/conclusion).	`medium`
`duplicate_content`	Un bullet cuyas palabras clave ya aparecieron en otra slide.	`medium`

5.11.4. Revisor LLM

- `_llm_quiz_issues` / `_llm_slide_issues`: una única llamada por artefacto con `QUIZ_CRITIC_PROMPT` / `SLIDE_CRITIC_PROMPT`. El LLM responde un JSON `{"issues": [...]}` que se valida con `QuizIssue` / `SlideIssue`. Se filtran ids fuera de rango.

5.11.5. Refinamiento selectivo

- `refine_quiz(client, kb, questions, quiz_plan, max_iterations=1, use_llm=True)`:
  - Llama a `review_quiz` (determinista + LLM) y obtiene los `critical_ids` (severity `medium`/`high`).
  - Solo esos se **regeneran** usando la **misma** `PlannedQuestion` (se preserva concept/bloom/kind/focus → la distribución global no se rompe).
  - Las preguntas que pasaron la revisión se mantienen literales.

- `refine_slides(...)`: idéntico, pero por `slide_index`. La correspondencia se hace por `title` porque el índice cambia si se descartó alguna slide previa.

Coste extra por refinamiento: *K* llamadas = nº de elementos críticos + 1 llamada de revisor. Es mucho más barato que regenerar todo.

5.12. `src/quiz_pdf_exporter.py`

ReportLab + `SimpleDocTemplate` en A4 con márgenes 2 cm:

- Hoja 1: enunciado + 4 opciones (A/B/C/D) por pregunta.
- Salto de página.
- Hoja 2: respuesta correcta, nivel Bloom y justificación.
- Estilos propios (`_styles()`) en tonos universitarios (`#1f4e79`).

5.13. `app.py` — UI Streamlit

Estructura por secciones con estado en `st.session_state`:

- `_init_state` inicializa claves (`processed_pdf`, `kb`, `quiz`, `pptx_bytes`, bytes exportados, etc.).
- `render_sidebar` → selector de modelo + slider de preguntas + avisos de plantilla.
- `render_upload_section` → uploader con reset al cambiar de archivo, muestra métricas (páginas, caracteres, imágenes, tablas) y un expander con el Markdown extraído.
- `render_generate_section` → dos columnas; cada una valida preflight Ollama, asegura KB (la recalcula solo si no existía ya), y dispara el pipeline correspondiente. Excepciones mapeadas a mensajes accionables.
- `render_quiz_results` → `st.form` con `st.radio` por pregunta; al someter muestra aciertos y justificaciones y permite descargar JSON y PDF.
- `render_pptx_results` → previsualización del índice y los bullets, botón de descarga `.pptx`.

---

6. Modelos de datos y contratos

Resumen visual de todos los modelos Pydantic/dataclass en el proyecto:

```
ProcessedPDF (dataclass, frozen) ──────────── pdf_processor.py
  markdown: str
  num_pages, num_chars, num_images: int
  has_tables: bool

LiteralHints (dataclass) ──────────────────── extractors.py
  definitions: list[LiteralDefinition]
  code_blocks: list[LiteralCode]
  formulas:    list[LiteralFormula]
  key_terms:   list[str]
  .to_prompt_block(max_chars=3500) -> str

Chunk (dataclass, frozen) ─────────────────── map_reduce.py
  index: int; text: str
```

```
KnowledgeBase (BaseModel) ──────────────────── knowledge_base.py
  main_topic: str
  subtopics: list[str]
  definitions:  list[Definition]    # def:*
  examples:     list[Example]       # ex:*
  formulas_code:list[FormulaOrCode] # fc:*
  numeric_data: list[NumericDatum]  # dt:*
  relations:    list[Relation]      # rel:*
  conclusions:  list[str]

SlidePlan / PlannedSlide (BaseModel) ──────── plans.py
QuizPlan  / PlannedQuestion (BaseModel) ────── plans.py

Quiz / QuizQuestion / QuizOptions ──────────── quiz_generator.py
PresentationPlan / BuiltSlide (dataclass) ─── pptx_generator.py

QuizReview / QuizIssue ──────────────────────── critics.py
SlideReview / SlideIssue ────────────────────── critics.py
```

Los **prefijos de id** son estables y únicos: `def:*`, `ex:*`, `fc:*`, `dt:*`, `rel:*`. Todo el sistema descansa sobre esta convención.

---

7. Prompts

Centralizados en `src/prompts.py`. Decisiones transversales:

- **Todo en español** para mantener el registro académico hispanohablante.
- Las llaves literales de los ejemplos JSON se escapan con los tokens `@@OPEN@@` / `@@CLOSE@@`, que `_finalize()` reemplaza por `{{/}}` (llaves escapadas para `str.format`). Las constantes numéricas (`__MAX_BULLETS__`, `__MAX_CHARS__`, `__MIN__`, `__MAX__`, `__TITLE_LEN__`) se inyectan en el mismo paso.
- Cada prompt define: rol, objetivo, **reglas duras** numeradas, formato JSON exacto y, cuando aplica, ejemplos positivos/negativos.

Prompts activos:

Constante	Uso
`SYSTEM_EXPERT_ES`	System prompt común: “profesor universitario, precisión, no inventar”.
`MAP_SUMMARY_PROMPT`	Fase MAP: resumen por chunk.
`REDUCE_CONSOLIDATION_PROMPT`	REDUCE legacy a Markdown (se conserva para `consolidate_document`).
`REDUCE_TO_KB_PROMPT`	REDUCE moderno: consolida parciales + hints literales → KB JSON.
`SLIDE_PLAN_PROMPT`	Plan de slides con kinds cerrados; prohíbe `kind=conclusion`.

Constante	Uso
`SLIDE_BULLETS_FROM_ATOMS_PROMPT`	Redacción de bullets con reglas antiellipsis, antimeta, antiinvención de ejemplos, plantillas por kind y ejemplos positivos/negativos.
`CONCLUSION_FROM_KB_PROMPT`	Conclusión final con las mismas reglas anti-meta y anti-verbatim.
`QUIZ_PLAN_PROMPT`	Distribución Bloom por rangos (15-25 %/20-30 %/...) + mapa Bloom→kinds.
`QUIZ_QUESTION_PROMPT`	Redacción de UNA pregunta; reglas de longitud de opciones (±30 %), prohibiciones (`Todas las anteriores`, etc.), escenarios para niveles altos de Bloom, ejemplos positivos/negativos.
`QUIZ_CRITIC_PROMPT`	Revisor LLM del quiz (tipos de issue, severidades).
`SLIDE_CRITIC_PROMPT`	Revisor LLM de slides (tipos de issue, severidades).
`OUTLINE_PROMPT`, `SLIDE_CONTENT_PROMPT`, `CONCLUSION_PROMPT`, `QUIZ_GENERATION_PROMPT`	Prompts legacy de la v1 (mono-shot), conservados para fallback.

8. Instalación y ejecución

8.1. Requisitos previos

- Python 3.11+ (recomendado 3.12).
- **Ollama** instalado y corriendo: https://ollama.com/download
- Al menos un modelo descargado. Recomendado por defecto: `ollama pull qwen2.5:7b`. Otros soportados por la UI: `gemma2:9b`, `gemma3:4b`, `mistral:7b`, `llama3.2:3b`.
- GPU NVIDIA ≥6 GB VRAM recomendable. Con 12 GB (RTX 3060/4070) se alcanza fluidez en `qwen2.5:7b` y `gemma2:9b`.

8.2. Instalación

```
cd tfg-local
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

8.3. Lanzar Ollama con GPU

Si el launcher de bandeja lanza el servicio en CPU, cerrarlo desde el tray y arrancarlo manualmente:

```
# En una terminal independiente
$env:OLLAMA_LLM_LIBRARY = "cuda_v12"   # fuerza CUDA
ollama serve
```

Verifica con:

```
ollama ps    # 100% GPU debe aparecer en la columna PROCESSOR
```

8.4. Lanzar la aplicación

```
streamlit run app.py
```

Abre `http://localhost:8501` y sigue el flujo: subir PDF → generar Quiz / Presentación → descargar.

8.5. Troubleshooting frecuente

Síntoma	Causa / solución
`model 'qwen2.5' not found (status 404)`	Falta el tag. Usar siempre `qwen2.5:7b`. El cliente ya valida con match exacto.
`Error: listen tcp 127.0.0.1:11434: bind: Only one usage…`	Ya hay un `ollama.exe` corriendo. Cerrar desde bandeja + `Get-Process ollama | Stop-Process -Force`.
`ollama ps` muestra `100% CPU`	Forzar `$env:OLLAMA_LLM_LIBRARY="cuda_v12"` antes de `ollama serve`.
`ScannedPDFError`	PDF sin capa de texto. Pre-procesa con `ocrmypdf`.
`QuizPlan vacío tras sanear`	El LLM no produjo concept_ids reconciliables. Revisar la KB (a veces los `rel:*` resultan muy abreviados); reducir el número de preguntas o cambiar de modelo.

9. Historial de mejoras (changelog técnico)

Cada entrada lista **qué se hizo**, **por qué** y **qué archivos cambiaron**. A partir de ahora se amplía con cada nueva intervención.

v1.0 — Base funcional inicial

- Pipeline “mono-shot”: PDF → Markdown → resumen Markdown libre → generación directa de quiz y de slides en una sola llamada cada uno.
- Exportación de quiz a PDF con ReportLab.
- UI Streamlit con modelo seleccionable y slider de preguntas.

- Detección de PDF escaneado con heurística de texto por página.

Limitaciones detectadas con uso real:

- Solapes entre slides (mismo concepto repetido).
- Distribución Bloom desequilibrada (todo `recordar`).
- Distractores triviales ("Ninguna de las anteriores", opciones absurdas).
- Bullets truncados con `...`, meta-lenguaje ("en esta diapositiva..."), copias literales del bloque de contexto.
- Conclusiones duplicadas (LLM añadía la suya y luego el renderer añadía otra).

v1.1 — Preflight y fix de modelo

- **`src/config.py`**: `AVAILABLE_MODELS` y `DEFAULT_MODEL` pasan a usar **tags completos** (`qwen2.5:7b`, `gemma2:9b`, `gemma3:4b`, `mistral:7b`, `llama3.2:3b`).
- **`src/ollama_client.py`**: `check_model_available` endurecido a match exacto. *Motivo*: `/api/generate` exige el nombre completo con tag; un match por base name superaba el preflight y después fallaba en tiempo de generación con 404.

v2.0 — Arquitectura por etapas (de resumen libre a KnowledgeBase)

Decisión estructural: abandonar el resumen en Markdown libre y adoptar una Knowledge Base JSON tipada como fuente única de verdad.

- **Nuevo:** **`src/knowledge_base.py`** con Pydantic de todos los átomos y `KnowledgeBase`. Implementado `_normalize_id` (`NFKD` + minúsculas + `ñ→n` + `[a-z0-9_-]` + `max_len=40`) aplicado como `@field_validator(id, mode="before")` en cada átomo → resuelve el `ValidationError` clásico al recibir `ex:farmacéutica` del LLM.
- **Nuevo:** **`src/extractors.py`** con heurísticas literales (definiciones, bloques de código, fórmulas, términos). El bloque resultante se inyecta en el prompt REDUCE para anclar la KB al texto original.
- **Nuevo:** **`src/plans.py`** con `SlidePlan` y `QuizPlan` como contratos intermedios, Bloom y kinds tipados con `Literal`.
- **`src/prompts.py`**: añadidos `REDUCE_TO_KB_PROMPT`, `SLIDE_PLAN_PROMPT`, `SLIDE_BULLETS_FROM_ATOMS_PROMPT`, `CONCLUSION_FROM_KB_PROMPT`, `QUIZ_PLAN_PROMPT`, `QUIZ_QUESTION_PROMPT`. Bug `KeyError` por llaves literales en el JSON de ejemplo resuelto con tokens `@@OPEN@@`/`@@CLOSE@@` procesados por `_finalize()`.
- **`src/map_reduce.py`**: nueva API `build_knowledge_base` que orquesta chunking + hints + MAP + REDUCE→KB. `consolidate_document` se conserva como shim de compatibilidad.
- **`src/quiz_generator.py`**: dividido en `plan_quiz` + `generate_single_question` (una llamada por pregunta con contexto central + contexto relacionado para distractores + preguntas previas para evitar solape).
- **`src/pptx_generator.py`**: dividido en `plan_slides` + `render_slide_bullets` + `_render_conclusion` + `render_pptx`.
- **`app.py`**: `_ensure_summary` → `_ensure_kb`; se pasa la KB a los generadores.

v2.1 — Revisor crítico y refinamiento selectivo

Nuevo: **`src/critics.py`** con dos pasadas:

- Determinista (regex + reglas de anclaje). Barato y detecta lo que el propio LLM no admitirá en sí mismo.

- LLM con `QUIZ_CRITIC_PROMPT` / `SLIDE_CRITIC_PROMPT` (una llamada por artefacto completo) para problemas semánticos.

`refine_quiz` / `refine_slides` regeneran **solo** los elementos con severidad `medium`/`high`, reutilizando la misma `PlannedQuestion`/`PlannedSlide`. Esto preserva la distribución global y abarata el coste (K llamadas extra en lugar de rehacer todo).

Integración:

- `generate_quiz(refine=True)` en `quiz_generator.py`.
- `build_plan(refine=True)` en `pptx_generator.py`.
- Import diferido (`from .critics import refine_*`) para evitar ciclos con los generadores.

v2.2 — Grounding y fuzzy matching

Problema recurrente: el LLM escribe `concept_ids` o `atom_ids` que no existen tal cual en la KB (acentos, plurales, abreviaturas tipo `rel:subclase_de` frente a `rel:subclase_de_bicicletademontana_bicicleta`). Resultado anterior: se descartaban preguntas enteras.

- **`src/plans.py`**: añadidos `_deaccent_lower`, `_normalize_slug` (plurales → singular), `_split_id`, `_collapse`, `resolve_atom_id` (cascada de 5 intentos) y `_reconcile_atom_ids`. **Restricción de prefijo**: la reconciliación solo busca dentro del mismo prefijo (`def:` solo con `def:*`), porque el resto del pipeline asume el tipo.
- **`sanitize_slide_plan`** y **`sanitize_quiz_plan`** usan la reconciliación y registran en log cada recuperación.
- **Deduplicación relajada**: `sanitize_quiz_plan` permite repetir `concept_id` mientras cambie al menos uno de `bloom_level` o `kind` → dos preguntas sobre el mismo concepto, una de `recordar` y otra de `aplicar`, son válidas.

v2.3 — Calidad visual de los bullets PPTX

Problema: bullets cortados con `…`, copias textuales del bloque de átomos del contexto (`def:xxx · definición · …`), meta-lenguaje, bullets demasiado cortos, conclusiones duplicadas.

- **`src/config.py`**: `MAX_CHARS_PER_BULLET` 140 → **180** (~2 líneas a 20 pt, permite densidad técnica).
- **`src/pptx_generator.py`**: añadidas tres utilidades de texto:
  - `_truncate_title(text, max_len)`: corte duro con `…` *solo* para títulos.
  - `_trim_by_words(text, max_len)`: recorte por palabra **sin** puntos suspensivos, cerrando con `.` si hace falta. Usado para bullets.
  - `_clean_bullet(text, max_len)`: descarta bullets con `…`/`...`, con prefijos de átomo (`def:`, `ex:`, ...), con patrones `· definición ·`/`· ejemplo ·`, con aperturas meta y con longitud <25 chars. Si todos los bullets de una slide se descartan, se lanza `GenerationError` para que el revisor dispare la regeneración.
- **`render_slide_bullets`** y **`_render_conclusion`** usan `_clean_bullet`.
- **`render_pptx`** filtra slides vacías y **construye el índice dinámicamente** solo con las slides que sobreviven (antes podía mostrar en el índice slides que no existían tras el filtrado).
- **`SLIDE_BULLETS_FROM_ATOMS_PROMPT`** y **`CONCLUSION_FROM_KB_PROMPT`** reforzados con reglas explícitas: longitud 12-28 palabras por bullet, prohibición de elipsis, prohibición de copia del bloque de

átomos, prohibición de meta-lenguaje, prohibición de inventar ejemplos fuera de la KB (p. ej. si la KB solo habla de "Bicicleta", no introducir "Coche"). Incluyen ejemplos positivos y negativos concretos.
- `SLIDE_PLAN_PROMPT`: explícitamente prohíbe `kind=conclusion` (la conclusión la añade el renderer).
- `sanitize_slide_plan`: descarta slides con `kind=conclusion` **o** título que empieza por `"conclus"` para evitar la duplicación aunque el LLM la cuele con otro kind.

---

10. Decisiones de diseño y *por qué* no se hizo de otra forma

- **Por qué KnowledgeBase y no un RAG con embeddings**. Los embeddings requieren un modelo adicional siempre cargado (más VRAM) y complican la auditoría. La KB explícita cabe en la ventana de contexto de un 7B y permite trazabilidad (cada bullet/pregunta apunta a un id). Un RAG se añadirá solo si pedimos dedup semántica global.
- **Por qué planificar antes de generar**. Generar en un shot provoca duplicados, drift y sesgo hacia `recordar`. Un plan tipado hace auditables la distribución Bloom, los conceptos cubiertos y los solapes.
- **Por qué prompts en español**. Mejor calidad sobre documentos académicos en español: el modelo evita calcos inglés→español en términos (especialmente en modelos 7B). Además el público objetivo de universidad hispanohablante exige el registro correcto.
- **Por qué `modelo:tag` exacto en la validación**. Ollama `/api/generate` exige el nombre completo; un falso positivo en el preflight se paga con error opaco durante la generación, difícil de diagnosticar.
- **Por qué fuzzy matching restringido al mismo prefijo**. Un `rel:algo_mal_escrito` solo tiene sentido como relación; permitirle resolver a un `def:*` rompería la coherencia del kind del slide o del kind de la pregunta. El filtro por prefijo es barato y elimina los falsos positivos.
- **Por qué `_clean_bullet` descarta en vez de arreglar los `…`**. Intentar "completar" un bullet truncado siempre termina en una frase sintéticamente mala. Es más limpio provocar `GenerationError` y dejar que el revisor regenere la slide completa con nuevo contexto.
- **Por qué el renderer siempre añade conclusión y no el LLM**. La conclusión "administrativa" es determinista (título fijo, bullets filtrados con las mismas reglas). Si también la añade el LLM, aparece duplicada; el saneador cierra esa puerta.
- **Por qué se conservan prompts legacy (`QUIZ_GENERATION_PROMPT`, `OUTLINE_PROMPT`, `SLIDE_CONTENT_PROMPT`, `CONCLUSION_PROMPT`)**. Sirven de fallback para depuración y como referencia. `consolidate_document` usa el REDUCE legacy para quien aún espere Markdown libre.

---

11. Roadmap pendiente

Orden sugerido, acordado con el usuario ("bien hecho, poco a poco, calidad"). Cuando se complete una entrada, muévela a la sección Historial de mejoras.

Bloque 3 — Calidad pedagógica (resto)

- ☐ **3.1** Enriquecer distribución Bloom: *verificación post-plan* + rebalanceo dirigido (ahora mismo la distribución la pide el prompt, pero no se fuerza post-hoc).
- ☐ **3.3** Plantillas por `kind` de slide más específicas (ahora hay reglas generales + mapa `BLOOM_RECOMMENDED_KINDS`; falta cada kind con su "esqueleto" narrativo).

Bloque 4 — Refinamiento avanzado

- ☐ **4.2** Deduplicación semántica con embeddings locales (opcional; actualmente se usa Jaccard de tokens y stem). Requiere modelo de embeddings en Ollama.

Bloque 5 — Material crudo

- ☐ **5.2** Speaker notes (notas del ponente) por slide, ancladas a la KB.

Bloque 6 — Mejoras visuales PPTX

- ☐ Uso de layouts variados de la plantilla (no solo 0 y 2).
- ☐ Sub-bullets cuando el contenido lo pide (p. ej. `kind=example` con atributos/métodos).
- ☐ Slide inicial de "conceptos clave" (glosario visual) a partir de `def:*`.
- ☐ Insertar imágenes del PDF en slides de `kind=example` cuando existan.

---

12. Convenciones de contribución y del README

1. **Este README es obligatorio de mantener**. Cualquier PR que añada o cambie comportamiento debe modificar como mínimo una de estas secciones: 5. Módulos y API interna, 6. Modelos de datos y contratos, 9. Historial de mejoras y, si procede, 7. Prompts y 11. Roadmap pendiente.
2. **Cada entrada del changelog** debe incluir: qué cambió, por qué, y los archivos afectados.
3. **Imports**: `from __future__ import annotations` en todos los módulos `src/*.py`; tipado con `typing.Literal` para vocabularios cerrados.
4. **Pydantic antes que dataclass** cuando hay validación o serialización JSON involucrada; `dataclass` para contenedores internos puros.
5. **Prompts**: siempre en `src/prompts.py`, escritos en español, con reglas numeradas, formato JSON exacto y ejemplos positivos/negativos cuando el LLM tiende a fallar.
6. **Logs**: `logger = logging.getLogger(__name__)` por módulo. `INFO` para eventos normales, `WARNING` para decisiones de fallback (id descartado, slide omitida), `ERROR` para fallos que interrumpen.
7. **Errores de dominio**: usar las clases de `src/exceptions.py`. No lanzar `Exception` genérico.
8. **Importaciones circulares**: si un módulo del revisor necesita un generador, usar `TYPE_CHECKING` para type hints y **import diferido dentro de la función** para el runtime (patrón ya aplicado en `critics.py`).